

AI: Artificial Intelligence

Date 24<sup>th</sup> Sep, 24

OBE

outcome based Education

↳ CLO

course learning outcome

performance = accuracy, response time.

LLM's = Large Language Models.

Modeling

Inference

• Separate

• Right now

• Paper is the

them

we are

inference. (Testing)

both.

learning it

is modeling. (knowledge representation)

Knowledge-based system:

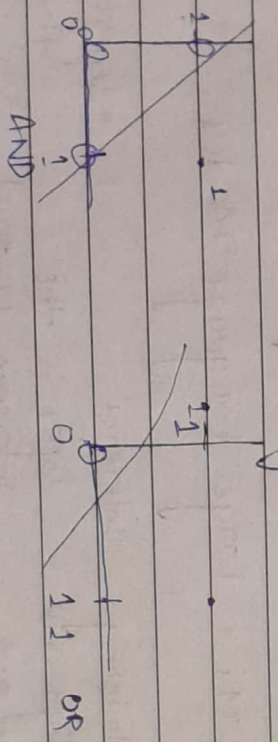
Represent or describe knowledge

for the machine to learn from it.

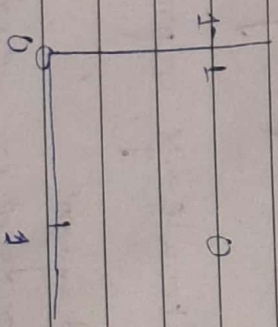
DECENT

Previous were logic based system  
 Artificial Neural Network  $\leftarrow$  Inspired by human brain.

Regions on graph that can be separated by a line are called linearly separable.



Non-Linear.



Backpropagation: Neural networks containing multiple layers.

\* First practical application of deep learning. (convolutional neural networks) to recognize handwritten digits.

Neural data hungry  
 Networks need a lot of data computation

LECTURE # 02

AlphaGo  $\rightarrow$  logic Tradition  $\rightarrow$  Neural

AI agents AI tools  $\rightarrow$  McCarthy  $\rightarrow$  Neural

we create intelligence  $\rightarrow$  How can we benefit society.



Date \_\_\_\_\_

• Procedural knowledge

• Declarative knowledge

→ Data biasness results in model biasness.

• Machines

require millions of examples to learn or function

Paradigm

Modeling

Inference

learning

• During modeling some relevant information may also be misplaced & only relevant data is used extra data may increase its complexity.

• Inference is used to get the best / shortest possible path. (test)

• learning is the training of the model using diff data.

DECENT

Date \_\_\_\_\_

Low level

intelligence

Machine

learning

High level

intelligence

Machine learning:

Data → Model

• Data driven models.

• real world complexity shift from code to data.

• Requires a leap of faith: generalization (It answers accurately on unknown data too).

• Reflex based models.

• One model has learnt fixed

computation takes place during inference.

• State based models

e.g. chess.

• After every move state is changing  
• order of imp

DECENT

Date \_\_\_\_\_

Markov decision processes:

→ uncertainty

Search problem:

Adversarial game:

→ full control

→ opponent is present.

Constraint satisfaction problem: no data

↳ order not imp. game

Marks distribution:

Quizes 15%.

Assignments 5%.

Project 10%.

Mid 20%.

Final 50%.

Optimization:-

Discrete Optimization:- find the best

discrete object. **DECENT**

Date \_\_\_\_\_

min cost (p)  
parameters

Algorithmic tool: dynamic programming

Continuous Optimization:- find the best  
vector of real numbers.

min Training Error( $w$ )  
 $w \in \mathbb{R}^d$

Algorithmic tool: gradient descent.

What is the objective/goal that  
needs to be achieved?

LECTURE #03 1<sup>st</sup> Oct, 24

Reflex Models:-

> linear predictors

> loss minimization

> stochastic gradient descent

**DECENT**

Date \_\_\_\_\_

Date \_\_\_\_\_

## LINEAR PREDICTORS:-

relates to our experience

Predictions can be divided into diff types

Classification Problems

Regression Problems

• Output is discrete value, either 0 or 1 (binary).

eg. gender

experience in form of price

recognition

by id

• for object

classification

each obj is

assigned a discrete

no.

• facial recognition

of emotions / expressions.

• Output is a real value.

Machine to

data provide

or discount

in this product

more data in future.

more experience.

december

prediction

values will be

real. **DECENT**

> Output having only two possible

only is binary

classification.

> More than one

two outputs is

called multi-

classification.

> Predictors are

made by looking at the problem

which classification it belongs to.

Spam classification:-

> first classification problem.

eg email automatically sent to spam

backend pay date predictor

language that analyze whether

that email is spam or not

> content, id, or sender can be

checked for analysis.

> output is either 0 (spam) or 1 (non-spam).

**DECENT**

Date \_\_\_\_\_

What is input in the email?

↳ email message: from where or whom?

date, subject, body

> input is represented with 'x'.

> what is the input form e.g. above is in textual form

Output:

> represented by 'y'

> either spam or not spam.

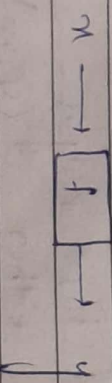
$y \in \{ \text{spam, non spam} \}$ .

Objective:

↳ Also predictor obtain for input x

↳ output y may change by looking at the data.

"To obtain a predictor f."



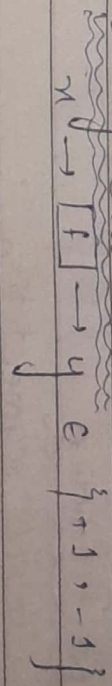
e.g. of binary classification

DECENT

Date \_\_\_\_\_

Types of Prediction tasks.

• Binary Classification



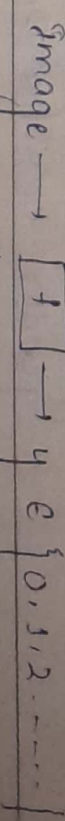
• Regression



e.g. size & location of the house  
predicting price of the house.

• Multi-class classification

Animals, vehicles.



> n different obj. results into n different outputs.  
> discrete nos.

• Ranking

e.g. players with their performance are ranked

> similar to multi-class

DECENT

Date \_\_\_\_\_

but generates the output in order.

### Structured Prediction

e.g. image segmentation

object in the image and its background pixels of both are classified this is called image segmentation.

all bits 0  
image black  
all bits 1  
image white

Task (T) → like spam classification  
" gender classification.

Experience (E) → represented through data.

### Data :-

Present in the form of diff examples:

cig 1 match 1 eg } of a cricket match  
10 match 10 eg }  
DECENT

Date \_\_\_\_\_

represented by  
example / instance →  $(x, y)$   
↓ ↓  
Input class label  
output / target

### Training Data:

- > It contains list of examples.
- > represented as  $(D_{\text{train}})$

$D_{\text{train}} = \left[ \begin{array}{l} \left( \text{" "}, +1 \right), \\ \left( \text{" usuy "}, -1 \right), \\ \vdots \end{array} \right]$   
Input content, whether spam or not (output)  
No. of examples

### Framework:

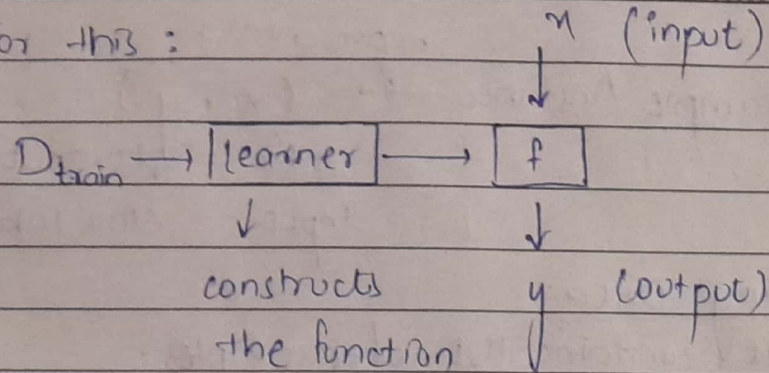
$D_{\text{train}} \rightarrow \left[ \begin{array}{c} x \\ f \\ y \end{array} \right]$

∴ To construct this function we need framework.

DECENT

Date \_\_\_\_\_

For this:



Lets focus on f :-

- It establishes a relationship with input to give output.
- It is based on feature extractor.  
e.g if sir want to predict my marks in AI based on the pre requisites of this subject.

This is the feature.

- In many cases features are provided but in some they are not.

DECENT

Date \_\_\_\_\_

- feature extractor, extracts the features from input. e.g date, name, subject, certain keywords in the email.

- Email needs to be brought into a feature so that feature can be extracted from it.

- FE (feature extraction) is an art, which depends on the task (prblm)

Feature Extractor:

e.g Predict why y, whether string x is an email address?

→ first you check what info or properties can you use as features e.g @, .com, .org.

→ What imp properties of x might be relevant for predicting y.

DECENT

Date \_\_\_\_\_

FE  $\rightarrow$  input:  $x$  (raw data)  
 output: set of features  
 (feature name, feature value)

e.g. abc @ gmail . com  $\rightarrow$  FE

- length > 10 of email address : 1
- fraction of alphabets : 0.85
- contains @ sign : 1
- ends with .com : 1
- ends with .org : 0

Feature vector: this feature info can be represented with vectors.

for computation we only use values: (not names)

represented with

|      |
|------|
| 1    |
| 0.85 |
| 1    |
| 1    |
| 0    |

DECENT

Date \_\_\_\_\_

Mathematically: features of the e.g.  
 $\phi(x) = \{ \phi_1(x), \phi_2(x), \dots, \phi_d(x) \}$   
 $\downarrow$   
 feature extractor

$\phi(x) \in \mathbb{R}^d \rightarrow d$ -dimensional vector.

$(x, y) \rightarrow$  is an example  
 $\downarrow$   
 $\phi(x) \rightarrow$  applying feature extractor  
 $\hookrightarrow (\phi_1(x), \phi_2(x), \dots, \phi_d(x))$  list of features in a single example.  
 $\downarrow$  input in features  $\downarrow$  output

In examples feature name might be same but their values differ.

| $D_{train}$ | length > 10 |  |  | $y$ |
|-------------|-------------|--|--|-----|
| 1           |             |  |  | +1  |
| 2           |             |  |  | -1  |
| !           |             |  |  |     |

each will have 100

DECENT

Date 3<sup>rd</sup> Oct, 24

## LECTURE #04

### • Weight Vector:

(Parameters)

• Main focus is on constructing  $\phi$

> for each feature 'j' we have a real no ' $w_j$ ' representing the contribution of feature for prediction

> has feature key against its weight key. (has its own same)

•  $\phi(x) \in \mathbb{R}^d$ ,  $w \in \mathbb{R}^d$

•  $\phi(x)$  depends on  $x$ .

•  $w$  does not depend on  $x$ .

e.g.  $w$   $\begin{cases} x_1 = \text{"Hello How are you"} \rightarrow \phi \\ x_2 = \text{"abc @xyz"} \rightarrow \phi \end{cases}$

• we have single  $w$  vector for all examples (Predictor).

\* weight vectors ki value agar **DECENT** mil gayi ham to mera function ban gaya

Date

|               |                                                            |              |                                                                 |              |
|---------------|------------------------------------------------------------|--------------|-----------------------------------------------------------------|--------------|
| $\phi(x_1) =$ | $\begin{bmatrix} -1.2 \\ 0.6 \\ 1 \\ 1 \\ 0 \end{bmatrix}$ | length $w =$ | $\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \\ w_5 \end{bmatrix}$ | if $w_j > 0$ |
|---------------|------------------------------------------------------------|--------------|-----------------------------------------------------------------|--------------|

• all length feature in all examples correspond to  $w_1$ , similarly all fraction of alphabet in all values (examples) corresponds to  $w_2$  and so on.

### Binary feature vector:

|             |                                                       |                          |
|-------------|-------------------------------------------------------|--------------------------|
| $\phi(x) =$ | $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ | only two possible values |
|-------------|-------------------------------------------------------|--------------------------|

• feature ki value 1 & weight is +ve then  $y = +1$  kiny main help krni go. **DECENT**

Date \_\_\_\_\_

if feature is 1 &  $w$  is  $-ve$  then it produces output  $y = -1$

\* How Linear Predictor uses FV & WV:

|      |      |                    |      |                    |
|------|------|--------------------|------|--------------------|
| WV = | -1.2 | $\phi(x_1) =$      | 1    |                    |
|      | 0.6  |                    | 0.85 |                    |
|      | -1   | $\in \mathbb{R}^d$ | 1    | $\in \mathbb{R}^d$ |
|      | 2.2  |                    | 1    |                    |
|      | 1.4  |                    | 0    |                    |

> Has a corresponding weight is multiplying by corresponding FV by linear predictor.

$$\begin{aligned} \text{Score (Score)} &= W \cdot \phi(x) = \sum_{j=1}^d \phi_j(x) \cdot w_j \\ &= \phi_1(x) \cdot w_1 + \phi_2(x) \cdot w_2 + \dots + \phi_d(x) \cdot w_d \end{aligned}$$

Date \_\_\_\_\_

for above example score will be:  
score = 4.51

$\therefore$  jitni bari value aye gi utni confident kray ge kr class/ot belong kray gi.

$\rightarrow$  value represents the confidence of classification as  $y = +1$ .

(No power or degree) (linear product / degree of polynomial is 1)

LINEAR PREDICTOR:

weight vector  $w \in \mathbb{R}^d$  dimensions should be same  
feature vector  $\phi(x) \in \mathbb{R}^d \rightarrow$

Task :- Binary Classification  
(linear) binary classifier

$$f_w(x) = \text{sign}(\phi(x) \cdot w) \text{ OR } (w \cdot \phi(x))$$

parameters

Takes  $n$  as an input & has  $w$  parameters

Date \_\_\_\_\_

$$\phi(x) = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} \quad w = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} \rightarrow \text{take transpose for single value.}$$

$$f_w(x) = \text{sign}(\phi(x) \cdot w) = \begin{cases} +1 & \text{if } w \cdot \phi(x) > 0 \\ -1 & \text{if } w \cdot \phi(x) < 0 \\ ? & \text{if } w \cdot \phi(x) = 0 \text{ or other.} \end{cases}$$

formally 'f' is defined here.

binary classifier designed

### Geometric Intuition:-

- Binary classifier  $f_w$  defines a hyperplane with normal vector  $w$ .

2D line  
surface  
multi-dimensional space.

\* hyperplane depends on dimensions. usually use 2D & we think it will work for other also.

$$w \in \mathbb{R}^2 \rightarrow f_w \text{ defines line (hyperplane)}$$

DECENT

Date \_\_\_\_\_

2, 3 power represents no. of features.

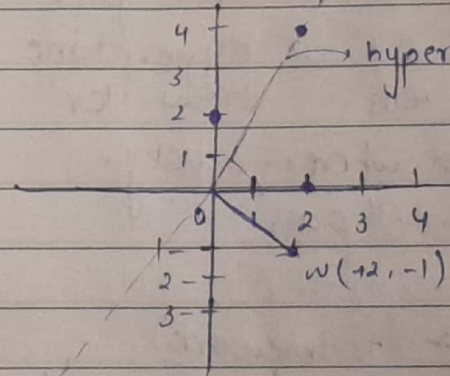
$$w \in \mathbb{R}^3 \rightarrow \text{plane}$$

Example: normal vectors

$$w = [-1, 2, -1] \rightarrow \begin{matrix} +1 & -1 & ? \end{matrix}$$

$$\phi(x) \in \{ [2, 0], [0, 2], [2, 4] \}$$

contains value of  $x$  not  $y$ .



hyperline/decision boundary.

normal vector & hyperplane where angle is less than  $90^\circ$  - those examples inside that angle are +ve & more than  $90^\circ$  are -ve.

$$w \cdot \phi(x_1) = [-1, 2, -1] \cdot \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}$$

$$= -4 < 0$$

$\rightarrow -1$

DECENT

Date \_\_\_\_\_

$$w \cdot \phi(x_1) = \begin{bmatrix} 2 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = -2 < 0 \quad \swarrow (-1)$$

$$w \cdot \phi(x_3) = \begin{bmatrix} 2 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = 0 \quad \swarrow (?)$$

$\hat{y} = mx + c$  → y-intercept  
 'jahan value zero ayegi wahan par'

$w \cdot \phi(x) > 0$  → hyperplane draw kr  
 this tells from where line will pass.

## LOSS MINIMIZATION:- (framework)

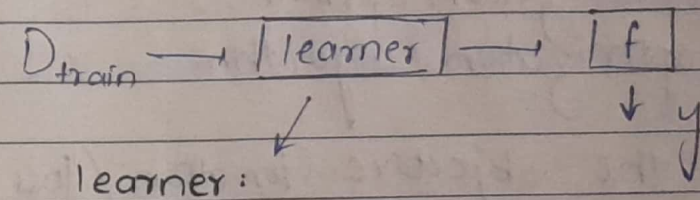
Modeling → function define kr liya  
 Inference ← Learning → how do you get parameters

DECENT

Date \_\_\_\_\_

"loss minimization uses learning as an optimization problem."

> on objective function is maximized or minimized to get an optimized output

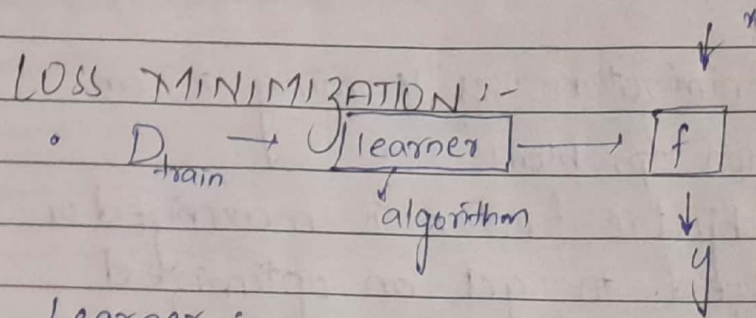


learner:

→ optimization problem is solved through optimization algorithm on which learner is based.

> error ko use kr kr loss function banaya ja skta, we use that loss to minimize errors by adjusting the weight  
 > it is an iterative process, try wo agli iteration main gholiyan nah hon

DECENT



Learner:

- optimization problem
- optimization algorithm

• What is the objective function/loss function?

- loss function is generally represented as:

$$\text{loss}(x, y, w)$$

input  $x$     output  $y$   
 ↓  
 current value of  $w$

$$\hat{y} = f_w(x) = \text{sign}(w \cdot \phi(x))$$

predicted class label

DECENT

input e.g

$x$   
abc @ ...  $w \cdot m$

class label / -target / actual output  
Date

ground truth

$y$

+1

already present

$$\phi(x) = \begin{bmatrix} - \\ - \\ - \end{bmatrix}, w = \begin{bmatrix} - \\ - \\ - \end{bmatrix}$$

after multiplying it gives predicted class label.

$$\text{loss}(x, y, w) =$$

value 0 accurate prediction

value big inaccurate prediction

• Two points:

- score =  $w \cdot \phi(x)$

- margin =  $(w \cdot \phi(x)) \cdot y$

either +ve or -ve ground truth

• jab bhi  $w \cdot \phi(x)$  and  $y$  have same sign prediction correct (+ve) if diff then prediction incorrect (-ve).

DECENT

Date \_\_\_\_\_

Input Example:

$$\phi(x) = [2, -1] \quad y = -1$$

$$W = [2, 0]$$

learner  
gives this

take transpose  
of  $w$  bcz it is in  
column.

$$\begin{aligned} f_w(x) &= \text{sign}(W \cdot \phi(x)) \\ &= \text{sign}([2, 0] \cdot [2, -1]) \\ &= \text{sign}([2, 0], [2]) \\ &\quad [-1] \end{aligned}$$

$$= \text{sign}(+4) > 0$$

score  $\hat{y} = +1$  (Incorrect prediction)

$$\text{margin} = (W \cdot \phi(x)) \cdot y = (-1)$$

$$= -4 < 0$$

Incorrect

prediction DECENT

Date \_\_\_\_\_

Loss Function:-

Defining this function for binary classification.

$$f_w(x) = \text{sign}(W \cdot \phi(x))$$

0-1 loss function

→ 0 correct prediction

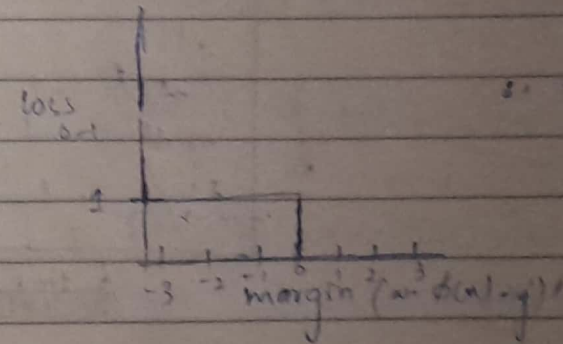
→ 1 incorrect prediction

0 → condition false  
1 → condition true

loss on single  
example (x, y)

$$\text{loss}(x, y, w) = 1 - f_w(x) \cdot y$$

$$= 1 - [(W \cdot \phi(x)) \cdot y \leq 0] \quad \text{Indicator function}$$



∴ Margin is value less than 0 loss will be one

DECENT

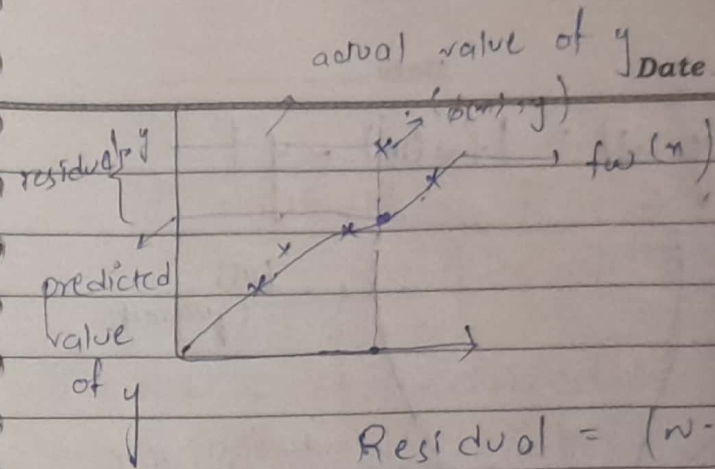
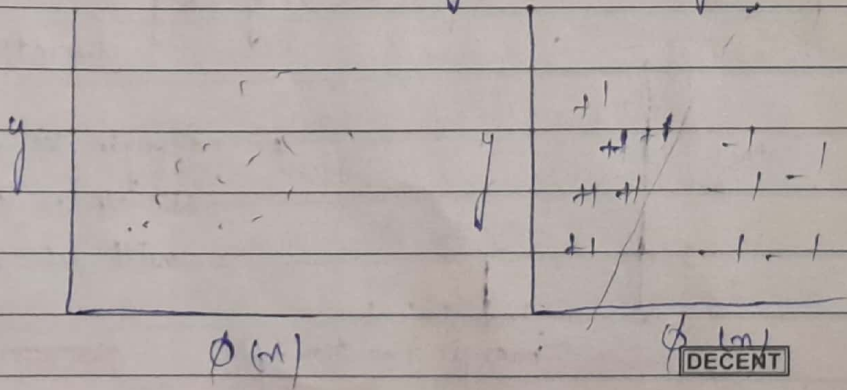
Date \_\_\_\_\_

## Loss function for Regression Problem.

- $f_w(x) = w \cdot \phi(x) \in \mathbb{R}$  - sign not included in this. bcz now every value is real value
- we cannot use 0-1 loss bcz there are infinite values that can come as output and they might not be 100% accurate to 0 or 1.

$$f_w(x) \in \mathbb{R}, y \in \mathbb{R}$$

$$f_w(x) = y \quad ? \quad \text{Very unlikely}$$



$$\text{Residual} = (w \cdot \phi(x)) - y$$

$$= f_w(x) - y$$

basic quantity that can be used to calculate loss regression.

$$\text{Loss} = (y - \hat{y})^2$$

(m, y)

Two types of loss func used:

→ \* Squared Loss Function

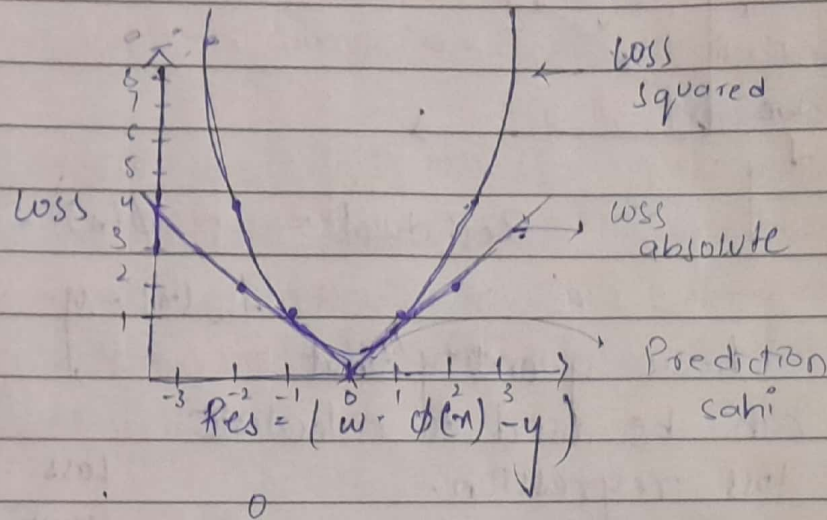
$$\text{loss}_{sq}(x, y, w) = (f_w(x) - y)^2$$

→ Absolute Deviation loss Function

DECENT

Date \_\_\_\_\_

$$\text{Loss}_{\text{absol}}(x, y, w) = |f_w(x) - y|$$



Loss Minimization Framework:-

Main objective:

(minimize training loss.)

accumulate loss on all data

Train loss (w)

depends on (w)

DECENT

Date \_\_\_\_\_

$$\text{Train loss}(w) = \frac{1}{n} \sum_{(x,y) \in D_{\text{train}}} \text{loss}(x, y, w)$$

optimization problem.

This loss is parametrized by  $w$ .

Minimize  $\text{Train loss}(w) \rightarrow$  overall data param loss.

$w \in \mathbb{R}^d$   
taking hamara model generalized hon.

Optimization Algorithm

\* Gradient Descent Algorithm

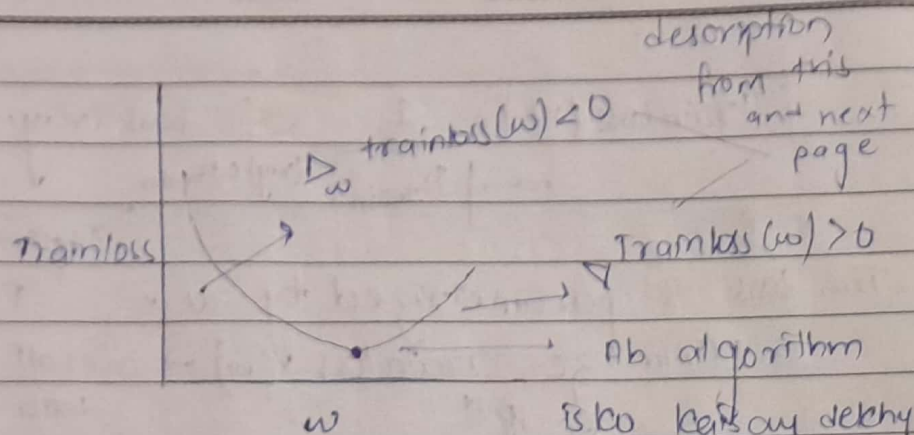
All deep learning models, chatbots & LLMs are based on this model.

→ Assume  $w \in \mathbb{R}$  → scalar value

→ plotting train loss on  $y$ .

DECENT

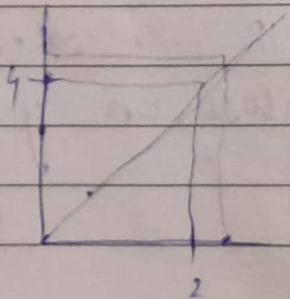
Date \_\_\_\_\_



→ gradient / slope / derivative ga

The gradient  $\nabla_w \text{Train loss}(w)$  is the direction that increases the loss the most.

$$\text{gradient} = \frac{dy}{dx} = \frac{y_2 - y_1}{x_2 - x_1}$$



$$f(x) = 2x$$

$$x = 2 \quad f(x) = 4$$

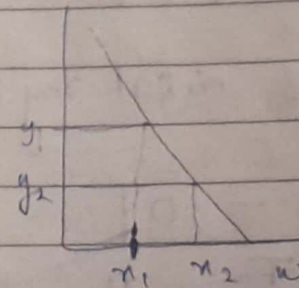
$$x = 2.002 \quad f(x) = 4.004$$

DECENT

Date \_\_\_\_\_

if it is a straight line the gradient will be same for each point of the line.

gradient  $> 0$



slope  $< 0$

gradient descent algorithm

$$W = W - \nabla_w \text{Train loss}(w)$$

Now when  $w$  value turn ans - main aye ga and opposite for -ve sign.

$$W - (-w) = w + w$$

so graph moves towards +ve side

Date \_\_\_\_\_

$$W = W - \eta \nabla_w \text{Trainloss}(w)$$

$\eta$  = learning rate  $< 1$   
 0.01 0.001  
 aise hi bar

main w ki value nikal rhy hain

- Initialize  $W = [0, 0, \dots, 0]$

- For  $t = 1 \dots T$  It is the no of iterations.  
 loop

$$W = W - \eta \nabla_w \text{Trainloss}(w)$$

New value of  $w$  ←  $W$  (old value of  $w$ )  
 ←  $\eta$  (learning rate)  
 ←  $\nabla_w \text{Trainloss}(w)$  (gradient)

$$\nabla_w \text{Trainloss}(w) = ?$$

Here needs to be a loop to accommodate training loss

DECENT

Date \_\_\_\_\_

• In each iteration we have to go through all examples and then update  $w$

→ Batch Gradient Descent ← very slow

## LECTURE # 06

22<sup>nd</sup> Oct, 24

For Gradient descent to work we have to first:

- Initialize  $W = [0, 0, \dots, 0]$
- For  $t = 1, 2, \dots, T$

(At each iteration you will see what is the loss computed resulted in -train loss).

- $\text{Trainloss}(w)$  → combining trainloss of all examples.
- $\nabla_w \text{Trainloss}(w)$
- $W = W - \eta \nabla_w \text{Trainloss}(w)$

→ to update the weight based on the trainloss.

DECENT

Date \_\_\_\_\_

Train loss (-)  $\rightarrow$   $w$  (+)" (+)  $\rightarrow$   $w$  (-)

### Batch Gradient Descent:

- Our weight will be updated after passing through all examples, then we
- updation is slow. compute loss & update the weight

### Stochastic Gradient Descent:

- relatively fast
- after every e.g. weight is being updated.

•  $w = [0, 0, \dots, 0]$ for  $t = 1, 2, \dots, T$ 

↑ example

for each  $(x, y) \in D_{\text{train}}$ we get  $\hookrightarrow$  Loss  $(x, y)$  $\nabla_w \text{Loss}(x, y)$ 

DECENT

Date \_\_\_\_\_

$$w = w - \eta \nabla_w \text{Loss}(x, y)$$

$$\text{Train Loss}(w) = \frac{1}{m} \sum_{(x, y) \in D_{\text{train}}} (\hat{y} - y)^2$$

$\underbrace{m}_{\text{No of training ex}} \quad \underbrace{(x, y)}_{\substack{\text{our prediction} \\ \text{with ground truth}}} \quad \underbrace{(\hat{y} - y)^2}_{\text{loss}}$

Taking Derivative:

$$\nabla_w \text{Loss}(x, y) = \nabla_w (w^T x - y)$$

$$= \nabla_w (w \cdot x - y)$$

$$\boxed{\nabla_w \text{Loss}(x, y) = 2(w \cdot x - y) \cdot x}$$

### • Step Size OR Learning Rate:

- Normally its value is used very small b/w 0 & 1 usually close to 0.

The more close it is to 1 more bigger steps you are taking

DECENT

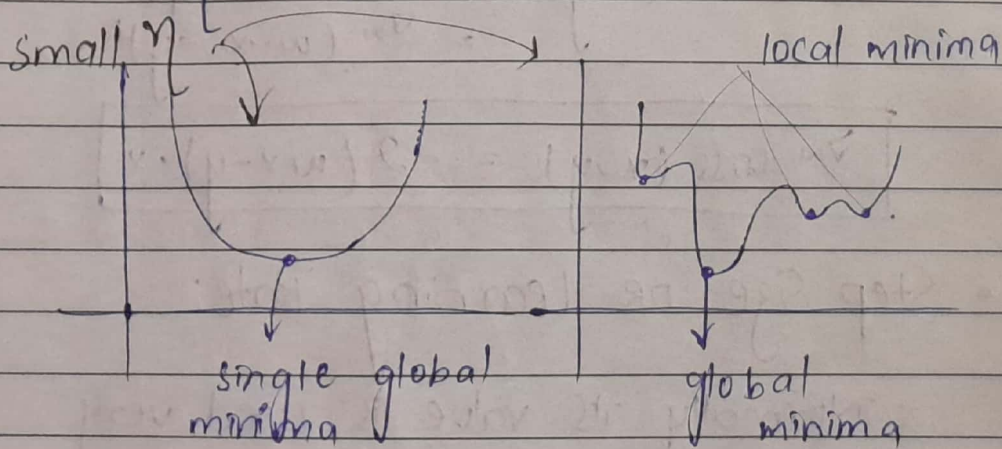
Date \_\_\_\_\_

and bigger would be the change in weights ( $w$ ), resulting in shooting global minima.

$$\eta = 0 \sim 1$$

$$\eta = 0.1, 0.01, 0.001$$

fixed values



It can also have adaptive values:

$\eta$  so for this in the beginning its value should be bigger and

DECENT

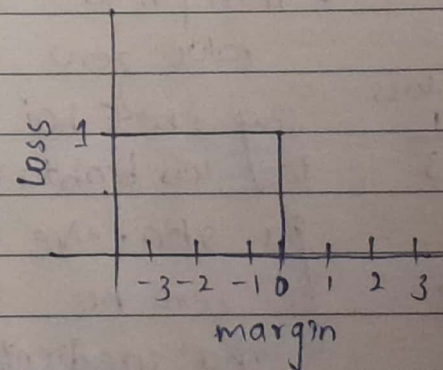
Date \_\_\_\_\_

then gradually decrease it.

$$\eta = \frac{1}{\sqrt{\text{No. of updates so far}}}$$

Regression  $\rightarrow$  Sq. loss  
Ab. Dev loss

Classification  $\rightarrow$  0-1 loss



This loss fun cannot be used in gradient descent bcz in g.d we calculate derivative here it is discrete.

• slope would be zero in most of the cases in 0-1 loss.

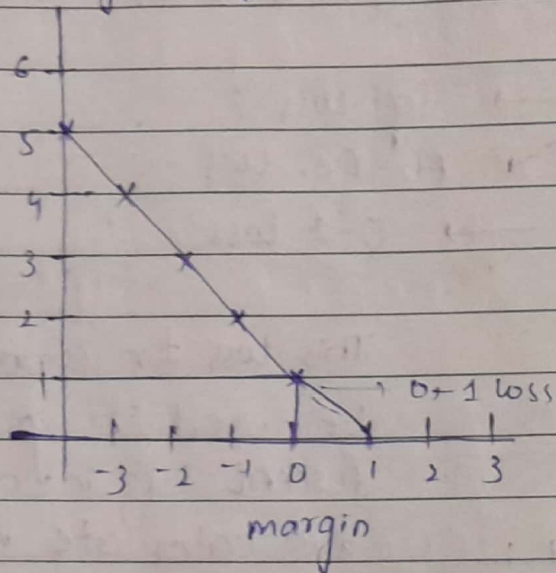
DECENT

Hinge loss:

$$\text{loss}_{\text{Hinge}}(x, y, w) = \max \left[ 1 - (w \cdot \phi(x)) \cdot y, 0 \right]$$

we get  
max b/w  
the ans &  
0.

• margin ki  
value zero  
say choti hai  
to loss badhta  
ja rha. +ve  
pe zero ho  
rha gradient



Now taking derivative  
of hinge loss

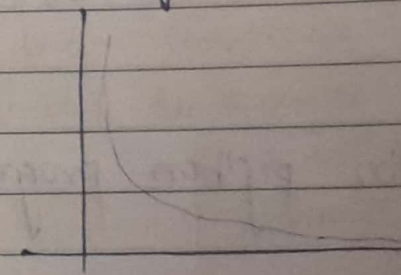
$$\frac{\partial}{\partial w} \text{loss}(x, y, w) = \begin{cases} 0 & (w \cdot \phi(x)) \cdot y > 1 \\ -\phi(x) \cdot y & \text{otherwise} \end{cases}$$

DECENT

In order for the loss gradient to  
not be just 0 for +ve margin but  
it keeps moving down we use  
logistic loss function.

Logistic Loss:

$$\text{loss}_{\text{log}}(x, y, w) = \log(1 + e^{-(w \cdot \phi(x)) \cdot y})$$



slope tilted so slope  $\neq 0$   
to  $w$  ki value change  
hoti hatti

SUMMARY

CLASSIFICATION

REGRESSION

Predictor

$\text{sign}(w \cdot \phi(x))$

score =  $w \cdot \phi(x)$

DECENT

$$w \cdot S(n) = y$$

$$(w \cdot S(n)) = y$$

Date

relate to  
correct y

margin

residual

loss fn

0-1 loss

square

hinge loss

absolute

logistic

Algorithm

BGD / SGD

BGD / SGD

Python:

google colab for python programming

integer division //

float division //

power \*\*

print (" ") / print (' ')

DECENT

Date

no data type is written for declaring

'in' operator → test whether a value is inside a container

0 } some  
-8 } index

x [1 : 4]

↑  
inclusive

↘  
exclusive

Collab

LECTURE# 07

24<sup>th</sup> Oct, 24

Python Functions

def funcname( ):  
statement  
statement

It is a fixed  
format language.  
indentation is  
imp.

• Must be declared above.

main code.

• calling should also be  
done after func definition.

input function takes input as a

DECENT

Date \_\_\_\_\_

string so if its an int input you have to typecast input.

- value in sequence } can be used as relational operator.

Mapping the equation into the programme for BGD.

We will run this programme on google collab.

points = [(2,4), (4,2)]

→ Cost fun / random function ground truth

def F(w):

return sum((w\*x - y)\*\*2 for x,y in points)

derivative

def dF(w):

return sum(2 \* (w\*x - y) \* x for x,y in points)

DECENT

Date \_\_\_\_\_

\* Gradient Descent

w = 0

eta = 0.01 (stepsize)

for t in range(100):

value = F(w)

gradient = dF(w)

computer loss calculating at loss derivative

(weight ← w = w - eta \* gradient  
updated) print('iteration {} : w = {}, F(w) = {}'.format(t, w, value))

in curly braces it prints.

$$\min f(w) = \frac{1}{16} (u-5) + 3$$

$$\min f(u) = (u-5) + 3$$

u=5 same ans

↑ Above programme is BGD.

Next e.g

DECENT

\* dimensions are features.

$w \cdot x$

Date \_\_\_\_\_

- Hum chaghty hai in kay data in form of matrix / a jaye / akey hum matrix manipulation ko (strain) (vectorize representation is preferred).

numpy  $\rightarrow$  python library for numerical computing.

$$f_w(x) = 3x$$

$\downarrow$   $w$       we       $\begin{array}{c|c} n & \\ \hline 1 & 3 \\ 2 & 6 \end{array}$

- There can also be data from generated data from this.

which we would have to get true  $w$ .

True  $w = \text{np.array}([1, 2, 3, 4, 5])$

$$f_w(x) = 1x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5$$

Assuming this

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

• dot product

DECENT

Date \_\_\_\_\_

- dot func used to get dot product

$$y = \text{true } w \cdot \text{dot}(x) + \text{np.random.randn()} \quad \text{if i comment}$$

- length say divide  $\frac{1}{D_{\text{train}}}$  - this that means we have perfect noise free data.

- $n \rightarrow D$  dimension

$$\underbrace{(w \cdot \text{dot}(x) - y)}_{\text{scalar value}} \times n \rightarrow \text{vector}$$

Taking derivative we respect to each  $w$ .

$$\nabla_{w_1} \cdot \text{TL} = \frac{2}{D} \sum (w^T x - y) \cdot x_1$$

$$\nabla_{w_2} \cdot \text{TL} = \frac{2}{D} \sum (w^T x - y) \cdot x_2$$

DECENT

Date \_\_\_\_\_

$$W = W - \eta \text{ Gradient} \quad \begin{matrix} (5 \times 1) \\ (5 \times 1) \end{matrix} \quad \begin{matrix} (1) \\ (5 \times 1) \end{matrix} \quad \begin{matrix} (5 \times 1) \\ \text{vector} \end{matrix}$$

Stochastic GD:

$i$  → training example  
vector value betwn kr rha  
 $m$  → no. of training examples

Saturation a skti hai!

LECTURE # 08

29<sup>th</sup> Oct, 24

Regression Example:

|          | $x$  | $y$ |                                              |
|----------|------|-----|----------------------------------------------|
| Input    | 1, 0 | 2   | supervised learning<br>if output<br>is given |
| features | 1, 0 | 4   |                                              |
|          | 0, 1 | -1  |                                              |

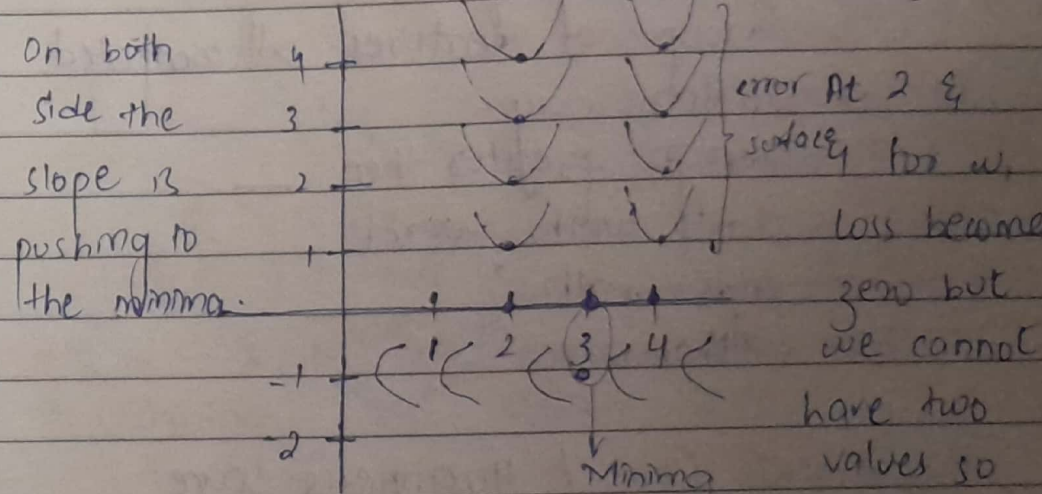
$$\text{loss } (n, y)^w = (w^T \cdot \phi(n) - y)^2$$

DECENT

Date \_\_\_\_\_

$$\begin{aligned} \bullet (w_1 + w_2 \times 0 - 2)^2 &= (w_1 - 2)^2 \\ \bullet (w_1 + w_2 \times 0 - 4)^2 &= (w_1 - 4)^2 \\ \bullet (w_1 \times 0 + w_2 \times 1 - (-1))^2 &= (w_2 - (-1))^2 \end{aligned}$$

$$\text{Trainloss}(w) = \frac{1}{3} [(w_1 - 2)^2 + (w_1 - 4)^2 + (w_2 - (-1))^2]$$



Batch Gradient is more robust, good but slow.

But in case of SGD the value would be either 2 or 4 and  $w$  value

DECENT

Date \_\_\_\_\_

would never be zero.

Features:-

- very imp in ML & AI

Feature templates:-

Group of features all computed in a similar way.

- length greater than \_\_\_\_\_
- starts with word \_\_\_\_\_
- ends with ? \_\_\_\_\_
- alphanumeric

Hypothesis Class / Hypothesis Space:

Predictor  $f_w(n) = w \cdot \phi(n)$  or  $\text{sign}(w \cdot \phi(n))$

It is the set of possible predictors with a fixed  $\phi(n)$  and varying

DECENT

If we change value of  $w$  it becomes new predictor. How many predictors can we make

$w$ :

$F = \{f_w : w \in \mathbb{R}^d\}$  This contains many predictors

Feature Extraction:  $\phi(n)$

These features tell how many predictors can be made using them by using domain knowledge for all features

All predictors (possible solution) we have to find which one is the best in this.

Learning:

find predictor  $f_w \in F$

based on data

Example:-

let  $\phi(n) = n$

$F = \{w_1 n + w_2 n^2 : w_1 \in \mathbb{R}, w_2 = 0\}$

hypothesis class

feature 1 feature 2

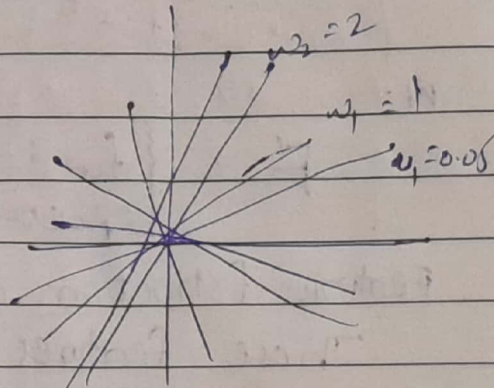
DECENT

Date \_\_\_\_\_

$$f_w(n) = n$$

$$f_w(n) = 2n$$

$$f_w(n) = 1.5n$$



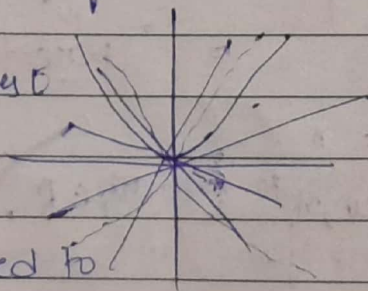
From here we have to choose the best.

Assume  $\phi(n) = \{n, n^2\}$

Second hypothesis class  $F_2 = \{w_1 n + w_2 n^2 : w_1 \in \mathbb{R}, w_2 \in \mathbb{R}\}$

Due to quadratic equation (value

it has largest set of predictors as compared to the first class.



DECENT

Date \_\_\_\_\_

Both straight line & quadratic curve can occur here.

$$F_1 \in F_2 \quad (F_1 \text{ belongs } F_2)$$

Features bahonay say hypothesis class ka size bahut jada hai.

Example Task:

Input  $x$ :

Two consecutive messages (whatsapp or email) in a chat.

Output  $y$ :

Whether second msg is a response to the first.

If it is target class 1 otherwise 0.

DECENT

Date \_\_\_\_\_

| $x$                                         |                                             | $y$ |
|---------------------------------------------|---------------------------------------------|-----|
| $m_1$                                       | $m_2$                                       |     |
| $\begin{pmatrix} - \\ - \\ - \end{pmatrix}$ | $\begin{pmatrix} - \\ - \\ - \end{pmatrix}$ | 1   |
| $\begin{pmatrix} - \\ - \\ - \end{pmatrix}$ | $\begin{pmatrix} - \\ - \\ - \end{pmatrix}$ | 0   |

For this we have to use feature extractor.

- Time elapsed —
- Time elapsed in b/w —

## LECTURE # 09

5<sup>th</sup> Nov, 24

- fixed features define a specific hypothesis class

one of the predictions  $\leftarrow f_w \in \mathcal{H}$  based on all data

DECENT

(Main literally so rahi aj pata nahi koo, is class main)

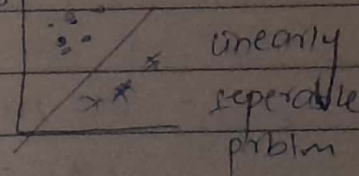
Date \_\_\_\_\_

- hypothesis class a best predictor is represented by  $g$ .
- overall global best  $f^*$ .
- Depending on the importance of feature your hypothetical class is either close to or far from global best.

## NEURAL NETWORKS:-

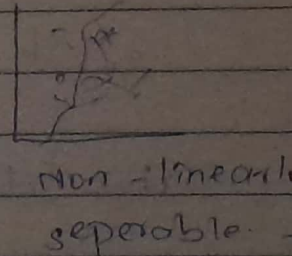
- It is a set of computation (fixed).
- Existing Models : Linear Models

Motivation { need non-linear models for non-linearly separable problem.



In traditional Machine Learning → we do

feature engineering manually.



→ Whereas in NN we learn features

DECENT

Neural networks are a bunch of linear functions stitched together with some non-linearities

Date \_\_\_\_\_

$\phi(x)$

Example (Predicting Car Collision).

Input: Position of two incoming cars

$$x = [x_1, x_2]$$

for first car

for second car

Output: Whether safe ( $y = +1$ ) or collide ( $y = -1$ ). distance is atleast 1 apart.

True function:  $y = \text{sign}(|x_1 - x_2| - 1)$

• If  $x_1$  &  $x_2$  position equal  $x_1 = x_2 \rightarrow$  will collide

• If  $x_1$  &  $x_2$  position diff  $x_1 \neq x_2 \rightarrow$  will not collide.

Data Set:

| $x$      | $y$  |
|----------|------|
| $[1, 3]$ | $+1$ |

DECENT

Date \_\_\_\_\_

$$[3, 1]$$

$$[1, 0.5]$$

$+1$

$-1$

How did we make indicator functions? It could be left or right

Decompose the problem. If car 1 is far right of car 2.

$$h_1 = 1 [x_1 - x_2 \geq 1]$$

Indicator function:  $h_1$  is equal to 1 if inside bracket is true

If Car 2 is far right of Car 1

$$h_2 = 1 [x_2 - x_1 \geq 1]$$

Sub-problems

So we will be safe if atleast 1 is true.

$$y = \text{sign}(h_1 + h_2)$$

DECENT

- \* In neural network there is always an extra feature in this case which is 1.

Date \_\_\_\_\_

Learning Strategy of this problem -

Define  $\phi(x) = \underbrace{\begin{bmatrix} 1, x_1, x_2 \end{bmatrix}}_{\text{positions of bias term}}$

Intermediate hidden subproblems.

$h_1 = 1[x_1 \cdot \phi(x) \geq 0]$  → general form of an indicator function  
where  $v_1 = [-1, +1, -1]$

$$\begin{aligned} v_1 \cdot \phi(x) &= -1 + x_1 - x_2 \\ &= x_1 - x_2 - 1 \geq 0 \\ \text{parameters} &= x_1 - x_2 \geq 1 \end{aligned}$$

$h_2 = 1[x_2 \cdot \phi(x) \geq 0]$

where

$$v_2 = [-1, -1, +1]$$

Final Prediction:

$$f_{v,w}(x) = \text{sign}(w_1 h_1 + w_2 h_2)$$

$$\rightarrow w = [1, 1]$$

DECENT

Learn both hidden subproblems  $v = (v_1, v_2)$  & combination of weights  $w = [w_1, w_2]$

Date \_\_\_\_\_

In this case the learner has to learn  $v$  &  $w$  both.  $v$  specifies the choice of hidden problem we are interested in.

Goal: Learn both hidden sub problems  $v = (v_1, v_2)$  and  $w = (w_1, w_2)$  (refined features)

| (Initial features) | $h_1$ | $h_2$ | $y$ |
|--------------------|-------|-------|-----|
| $x$                | 0     | 1     |     |
| $[1, 3]$           | 1     | 0     | +1  |
| $[3, 1]$           | 0     | 0     | +1  |
| $[1, 0.5]$         |       |       | -1  |

( $w$  specifies how do we take

GRADIENTS:- the results of hidden problems and come up with final

Problems, Gradient of  $h_1$  w.r.t  $v_1$  is 0 (prediction).

$$h_1 = 1[v_1 \cdot \phi(x) \geq 0]$$

Plotting:

DECENT

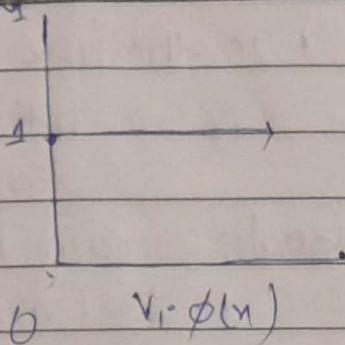
Date \_\_\_\_\_

→ logistic func:

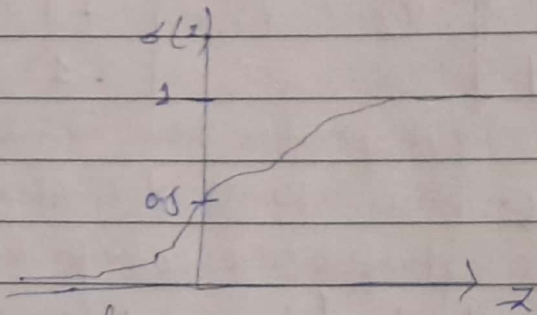
maps  $(-\infty, +\infty)$  to  $[0, 1]$

input  
can be  
" any

output  
will be  
this



• Function: (ask teacher)



Sigmoid function:-

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Derivative:

DECENT

Date \_\_\_\_\_

Now 
$$h_1 = \sigma(v_1 \cdot \phi(n))$$
  

$$h_2 = \sigma(v_2 \cdot \phi(n))$$

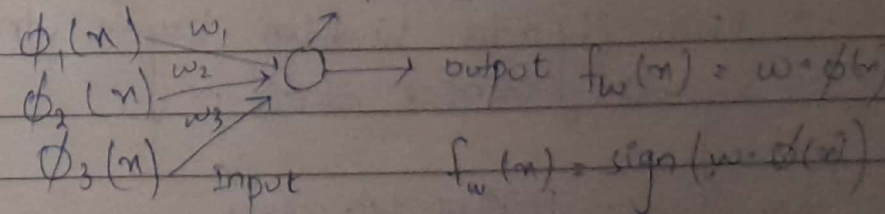
When value greater than 0.5 value true (1) If less than 0.5 it is zero.  
learning rate

$$v_1 = v_1 - \alpha \cdot \text{gradient}$$

$$v_2 = v_2 - \alpha \cdot \text{gradient}$$

Since gradient value will not be change zero so  $v_1$  &  $v_2$  will keep on changing

Linear functions:- Represented by neural network node/unit/neuron



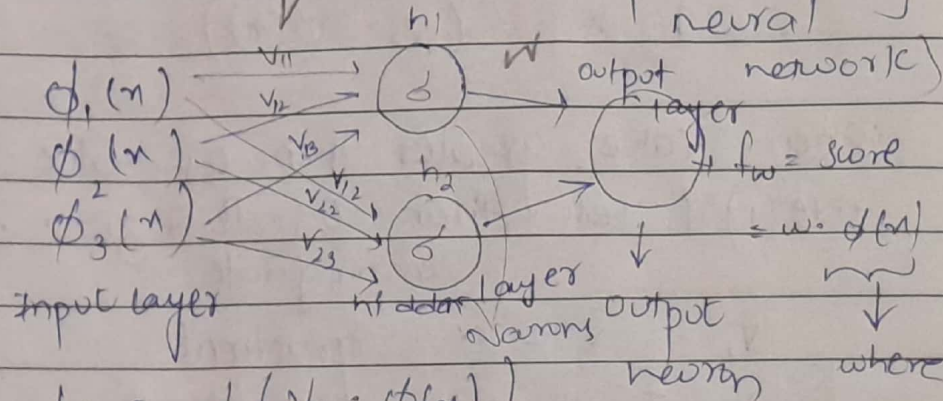
output  $f_w(n) = w \cdot \phi(n)$

$f_w(n) = \text{sign}(w \cdot \phi(n))$

DECENT

Date \_\_\_\_\_

Non-linear functions (Represented by neural network)



$$h_1 = \sigma(v_1 \cdot \phi(n))$$

$$h_2 = \sigma(v_2 \cdot \phi(n))$$

$$h_j = \sigma(v_j \cdot \phi(n)) = \sigma(2) = (1 + e^{-2})^{-1} \text{ where } (h_1, h_2)$$

Refined features which would be more good for final output.  $\text{score} = w \cdot h$

Jitni ziyada hidden layer utna ziyada refined data.

Key Idea: Feature learning.

$\phi(n)$  - manually specified features

DECENT

score =  $w \cdot h$

Date \_\_\_\_\_

$$h(n) = [h_1(n), h_2(n), \dots, h_k(n)]$$

automatically learned features  
→ both  $w$  &  $v$  are learning side by side.

$$\phi(n) \in \mathbb{R}^d \text{ (d-features present)}$$

$$h(n) \in \mathbb{R}^k \text{ (k-units/neurons present in hidden layer)}$$

$$v_j \in \mathbb{R}^d \text{ where } j = 1, 2, \dots, k$$

$$V \in \mathbb{R}^{k \times d} \text{ matrix}$$

$$V = \begin{bmatrix} 1 & v_{11} & v_{12} & \dots & v_{1d} \\ 2 & v_{21} & v_{22} & \dots & v_{2d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ k & v_{k1} & v_{k2} & \dots & v_{kd} \end{bmatrix}$$

$$w = [w_1, w_2, \dots, w_k]$$

DECENT

Date \_\_\_\_\_

## Loss Minimization:

Objective: Minimize  $\text{train loss}(v, w)$   
for Gradient

$$\text{Train loss}(v, w) = \frac{1}{|\text{Train}|} \sum_{(m, y) \in \text{Train}} \text{loss}(m, y, v, w)$$

for stochastic

$$\text{loss}(m, y, v, w) = \left( y - f_{v, w}(m) \right)^2 \quad \begin{array}{l} \downarrow \\ \text{ground truth} \end{array} \quad \begin{array}{l} \downarrow \\ \text{for regression problem.} \end{array}$$

$$f_{v, w}(m) = \sum_{j=1}^K w_j \cdot \underbrace{\sigma(v_j \cdot \phi(m))}_{\text{weights of the last layer}}$$

Goal:

$\nabla_{v, w} \text{Train loss}(v, w)$  → Go value is main say nitly  $g_i, w$  of  $\phi$  node  
no pass for  $d_i$

Computation Graph:

Approach to compute

gradient.

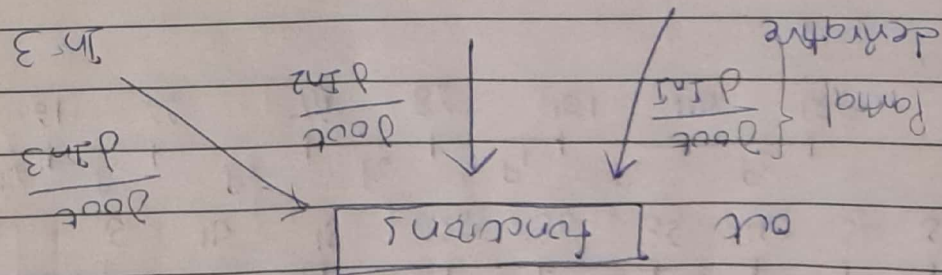
$$\sigma(v \cdot \phi(m)) = \frac{1}{1 + e^{-(v \cdot \phi(m))}}$$

(on OS last pages)

DECENT

(A3)

functions as boxes



Change in  $x_2$

$$\frac{d f_2}{d x_2} = \frac{d f_1}{d x_1} = \frac{d f_2}{d x_2} = \frac{d f_1}{d x_1}$$

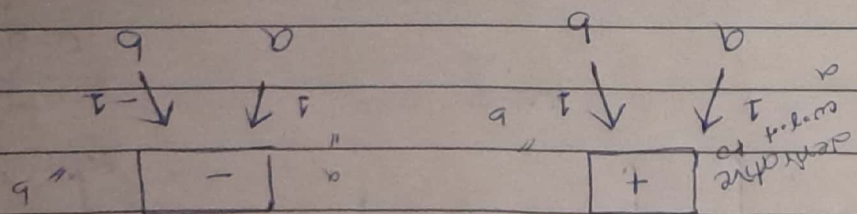
$$2 * x_1 + x_2 = f_1$$

$$2 * x_1 + (x_2 + \epsilon) * x_3 = f_2$$

$$\frac{d f_2}{d x_2} = \frac{d f_1}{d x_1} = \frac{d f_2}{d x_2}$$

$$\frac{d f_2}{d x_2} = \frac{d f_1}{d x_1} = 2$$

Basic Building Blocks:



$$out = a - b$$

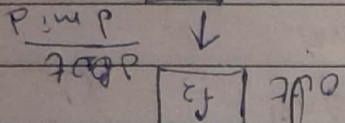
$$out = a + b$$

$$out = a * b$$

$$out = \max(a, b)$$

$$out = a * b$$

composing functions:-



$$\frac{d f_2}{d in}$$

$$\frac{d f_1}{d in}$$

$$d f_2 = d f_1 * d in$$

Date

Hinge loss:

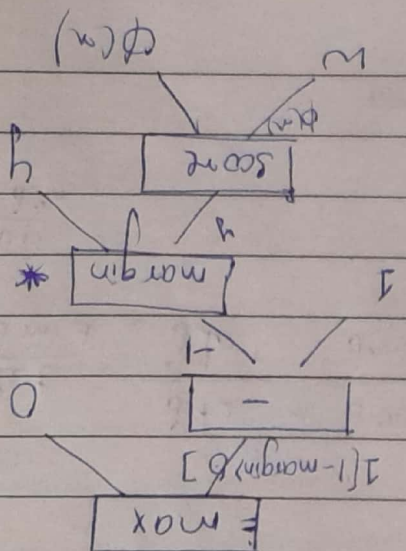
$$\text{loss}(y, w) = \max[1 - w \cdot \phi(x), 0]$$

Derivative:

$$\frac{\partial \text{loss}}{\partial w_1} = w_1$$

$$\frac{\partial \text{loss}}{\partial w_2} = w_2$$

On the side of  
line derivatives  
are taken



DECENT

of hinge loss w.r.t to  $w$   
(derivative)

Date

Gradient:

$$1 - [1 - \text{margin}] > 0 \cdot (-1) \cdot y \cdot \phi(x)$$

$$w = w - \alpha \cdot \text{Gradient}$$

In Neural Network

$$\text{loss}(y, w) = \sum_{i=1}^n y_i \cdot \phi(x_i) \cdot y_i$$

Assignment-1 (Deadline: Thursday - 08-20)

$$\frac{\partial \text{loss}}{\partial w_1} \quad \text{assume } k=2 \quad w=2$$

$$\frac{\partial \text{loss}}{\partial w_2}$$

$$\frac{\partial \text{loss}}{\partial w_1}$$

Using computation  
graph

Nearest Neighbor Classifier:-

Training: store training data (D\_train)  
Predicted  $f(x')$ :  
- find  $(m) \in D_{\text{train}}$  where  $\phi(m)$  is smallest

(in stars lost)  $\phi(m) \cdot \phi(x)$  is smallest

(AI)

Similar examples tend to have similar outputs.

Date \_\_\_\_\_

- return y

→ if any closely match then

$\| \phi(x) - \phi(x') \|$

gives a difference

$(\phi_1(x) - \phi_1(x'))^2 + (\phi_2(x) - \phi_2(x'))^2$  k-NN has a

extra bit of

value use

$$= \sqrt{\phi(x) - \phi(x')^2 + (\phi_2(x) - \phi_2(x'))^2} \quad g'$$

$$= | \phi_1(x) - \phi_1(x') | + | \phi_2(x) - \phi_2(x') | + \dots$$

go to nearest neighbour

Normal norm

on its value assigned to each neighbour

to f.

• This all is also called non-parametric method (KNN) or lazy learning.

DECENT

Date \_\_\_\_\_

KNN  $\rightarrow$  K nearest neighbour

Next topic would be

Unsupervised Learning (K-means)

$\hookrightarrow$  Input is provided only

DECENT

## LECTURE # 10

Date. 7<sup>th</sup> Nov, 24.

- $\sigma$  is added to add non-linearity to the output.
- If derivative is zero then it will be no learning that's why we use sigmoid function instead of step function that gives 0 or 1.  
 $\sigma(v \cdot \phi(m)) \rightarrow$  activation function  
due to which non-linearity comes.
- $f_w(m) = w \cdot \phi(m)$   
 $h = \sigma(v \cdot \phi(m))$   
 $f_w = w \cdot h$   
 $f_w(m) = w \cdot (\sigma(v \cdot \phi(m)))$

- In learning  $v$  &  $w$  are variables that are randomly initialized.

### GENERALIZATION:-

- Imp concept in reflex model:
- Usually in each case we have **DECENT**

Date \_\_\_\_\_

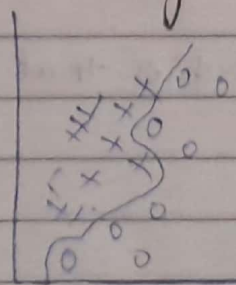
an objective function that we want to minimize.

True objective is that our algorithm gives accurate future prediction.

$$\text{Train loss}(w) = \frac{1}{|D_{\text{train}}|} \sum_{x_i, y_i \in D_{\text{train}}} \text{loss}(x_i, y_i, w)$$

↓ Is this our true objective? No.  
that our predictor is accurately predicting for the past data?

overfitting:



But in case of future data it gives (generalization) error.

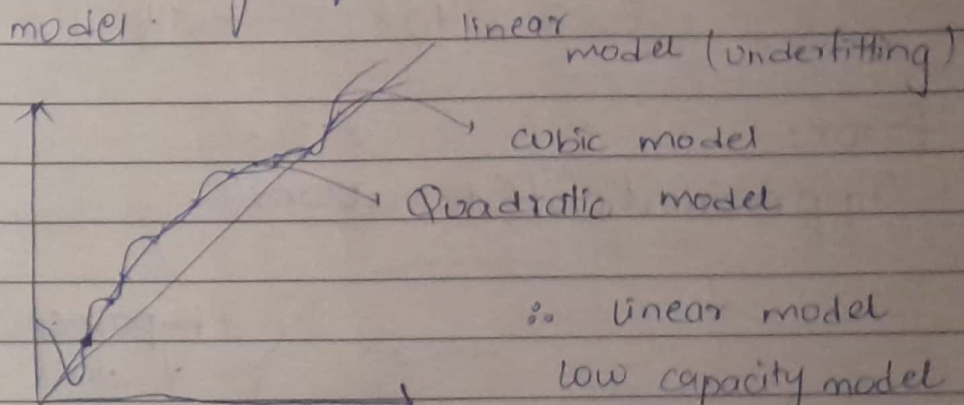
Training error  $\approx 0$   
Test / Generalization error  $\gg 0$

DECENT

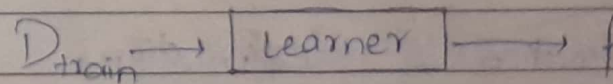
Date \_\_\_\_\_

This means our model is facing overfitting.

- We have to choose right model for right data eg quadratic model, cubic model.



∴ linear model  
low capacity model  
• Jitna high fair gay  
utni capacity bhi hogi.



How good is the predictor  $f$ ?

Goal: To minimize error on future (unseen) examples.

DECENT

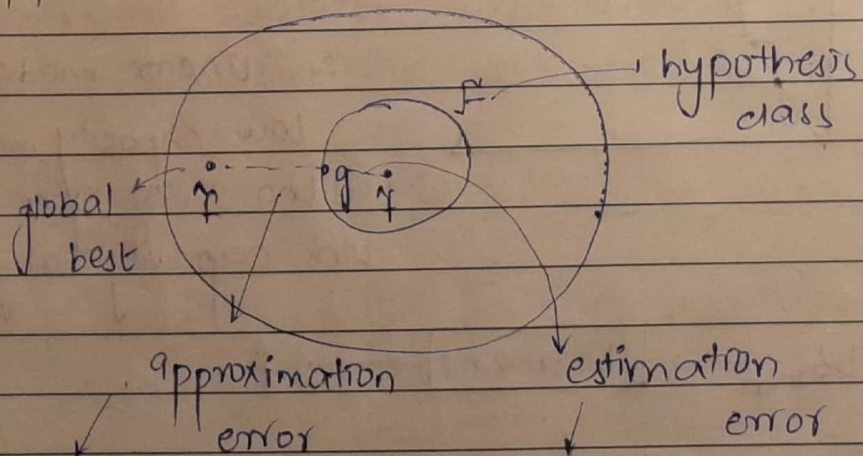
Date \_\_\_\_\_

| Data Set |      |  |
|----------|------|--|
| Train    | Test |  |
| 80 :     | 20   |  |
| 70 :     | 30   |  |
| 60 :     | 40   |  |

For checking performance of model after it has been built.

## Error:

### • Approximation/Estimation



∴ How good is the chosen hypothesis class?

∴ How good is your predictor which **DECENT** is learned relative

Date \_\_\_\_\_

### • Effect of Hypothesis Space / Class Size :

to the potential of hypothesis class.

If size increases → approx. error decreases

esti. error increases

How to control the hypothesis class size?

↓  
because searching would become very hard.

Strategy 1:

$w \in \mathbb{R}^d$  where  $d$  is no. of features

• First task is to reduce dimensionality.

∴ Manual Selection of features

- Add features if they help

- Remove features if they don't help.

Strategy 2:

Controlling the norm (length of the vector)

$w \in \mathbb{R}^d$

**DECENT**

Date \_\_\_\_\_

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_d^2}$$

for this we change our objective func.  
regularized objective func  $\rightarrow$  method 1

$$\min_w \underbrace{\text{TrainLoss}(w)}_{\substack{\downarrow \\ \text{Minimizing} \\ \text{training error}}} + \frac{\lambda}{2} \underbrace{\|w\|^2}_{\substack{\downarrow \\ \text{Try to keep} \\ \text{values of } w \text{ close} \\ \text{to zero}}}$$

overfitting: training error zero & future data is full of error.

underfitting: training error is large and future predictions simplified.

$\lambda$  - regularization parameter  
if  $\lambda = 0$  (lead to overfitting)  
if  $\lambda \gg 1000$  (lead to underfitting)

DECENT

Date \_\_\_\_\_

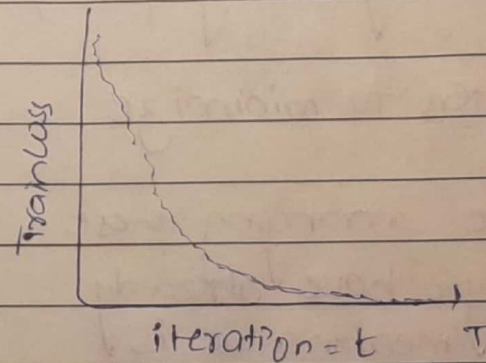
Early Stopping  $\rightarrow$  method 2  
In Gradient Descent:

$$w = [0, 0, \dots, 0]$$

for  $t = 1, 2, \dots, T$

$$w = w - \eta \nabla_w \text{TrainLoss}(w)$$

• Jitna jaldi stop  
utna kam khata  
of overfitting.



Train Model  $\rightarrow D_{\text{train}}$  (seen)  
Test Model  $\rightarrow D_{\text{test}}$  (unseen)

$v$ 's &  $w$ 's  $\rightarrow$  learnable values  
 $\eta, \lambda, T \rightarrow$  Hyperparameters  
features  $\rightarrow$

DECENT

Date \_\_\_\_\_

How to choose hyper parameters:-

- 1- Choose hyperparameters to minimize  $D_{train}$ ?  
 Sol:  $\lambda = 0$  ,  $T = \infty$  features = include all features  
 This will cause overfitting (wrong option)

- 2- Choose hyperparameters to minimize  $D_{test}$ ?  
 Sol: - If you are arranging these values then you have already shown test data meaning it is unbiased & unreliable (wrong option)

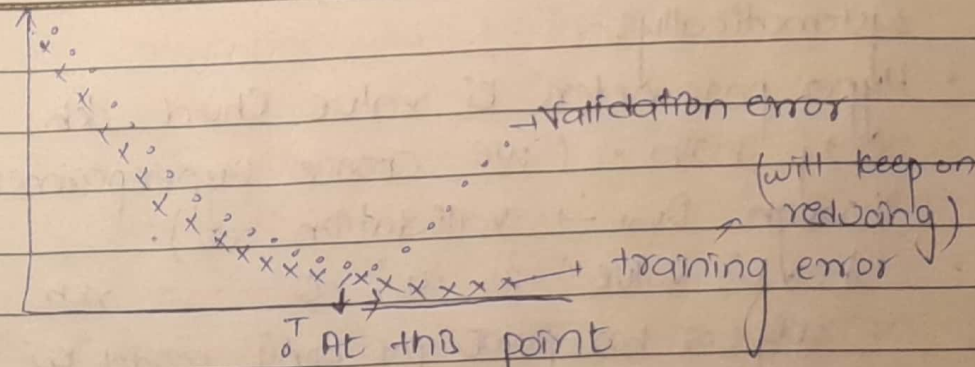
Problem: (Sol) ?  
 Data Set

|             |            |           |
|-------------|------------|-----------|
| $D_{train}$ | $D_{test}$ | $D_{val}$ |
|-------------|------------|-----------|

validation set used

to tune hyperparameter values DECENT

Date \_\_\_\_\_



|             |   |            |   |           |
|-------------|---|------------|---|-----------|
| $D_{train}$ | : | $D_{test}$ | : | $D_{val}$ |
| 70          | : | 15         | : | 15        |
| 80          | : | 10         | : | 10        |
| 60          | : | 20         | : | 20        |

LECTURE # 11

12<sup>th</sup> Nov, 24

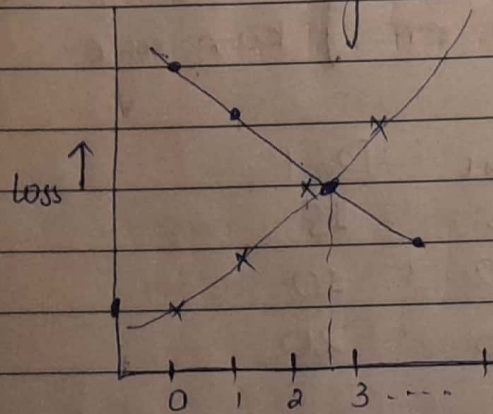
Normal parameters & values weights, v's are systematically updated DECENT

Date \_\_\_\_\_

systematically.

- Hyper parameters ki value khud rkh krni hai. (we choose hyperparameters through  $D_{val}$  → validation set).
- $\lambda$  ki value .2, .4, .6 ---- rkh

say or har point py apni model ko train krain gay or us ky hisab sy training loss calculate krain gay or validation loss bhi



$\lambda$  = regularization parameter

- x → training error
- o → validation error

DECENT

Date \_\_\_\_\_

$$\min \text{TrainLoss}(w) + \frac{\lambda}{2} \|w\|^2$$

- Training error  
bhtta (chala jaye ga) Taray Taray ye kam ho ga

- At 6 you see that instead of validation loss decreasing it is increasing so in this case best value for  $\lambda$  would be 5.

## • UNSUPERVISED LEARNING:-

- In supervised learning we are given input  $x$  & its corresponding output  $y$ .
- In supervised learning:

$D_{train}$  only contains inputs  $x$  unlabeled data. (normally it is cheaper)

- ziyada data → ziyada experience

Key Idea:

Data has lots of rich latent

DECENT

Date \_\_\_\_\_

structures.

- we want methods to ~~detect~~ discover these structures automatically.

Clustering:

similar characteristics  
to base pay existing  
data to groups  
main divide k-means

market segmentation  
e.g. apple new  
phone launch  
advertisement

Clustering:

Input: Training set of input examples

$D_{\text{train}}: \{x_1, x_2, \dots, x_m\}$   
 $m = \text{examples}$

Output: Assignment of each examples to a cluster.

$[z_1, z_2, \dots, z_n]$  where  $z_i \in \{1, 2, \dots, K\}$   
 $K = \text{no. of clusters}$

e.g.  $x_7$  belong to cluster 2 then  
 $z_7 = 2$ .

DECENT

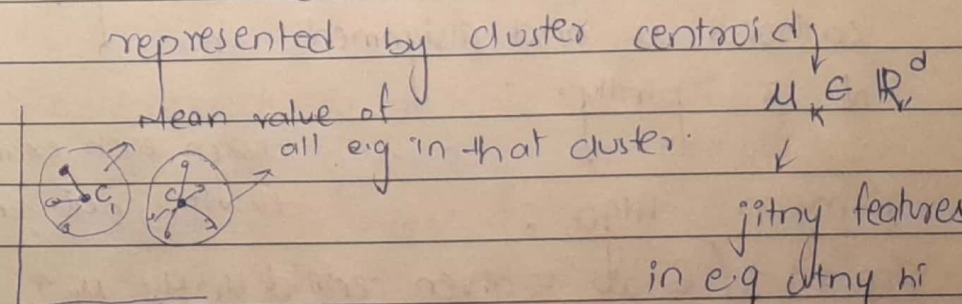
Date \_\_\_\_\_

Intuition: want similar examples to be in same cluster, dissimilar examples to be in a different clusters.

K-means Algorithm:

objective:-

- Each cluster  $K = 1, 2, \dots, K$  is represented by cluster centroid,



Intuition:

- we want each e.g.  $x_i$  close to its assigned centroid  $\mu_{z_i}$ .

DECENT

Date \_\_\_\_\_

Objective Function:-

$$\text{loss}(z, \mu) = \sum_{i=1}^n \|\phi(x_i) - \mu_{z_i}\|^2$$

loss cluster ka centroid kya hai har aik belong karta hai e.g. ka apni cluster sy distance kum sy kum ho.

- We need to choose centroid  $\mu$  and assignments of  $z$  jointly.

assign each example to the best centroid

K-mean Algo:

Step 1: (Goal: given centroid  $\mu_1, \mu_2, \dots, \mu_K$ )

- For each  $i = 1, 2, \dots, n$
- Assign  $i$  to cluster with closest centroid.

$$z_i = \underset{K=1,2,\dots,K}{\text{argmin}} \|\phi(x_i) - \mu_K\|^2$$

is t ky against wo value minimum mil rhi hai.

DECENT

\* Jitni centroid utni cluster.

Date \_\_\_\_\_

$$\phi(x_i) = \left\{ \phi_1(x_i), \phi_2(x_i) \right\}$$

$$\mu_K = (\mu_K^1, \mu_K^2)$$

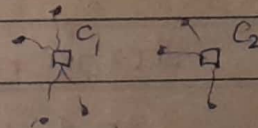
$$\|\phi(x_i) - \mu_K\|^2 = \sqrt{(\phi_1(x_i) - \mu_K^1)^2 + (\phi_2(x_i) - \mu_K^2)^2}$$

Step:- 2 (Goal: Given cluster assignment  $z_1, z_2, \dots, z_n$ , find the best centroid)

For each cluster  $K = 1, 2, \dots, K$   
set  $\mu_K$  to average of examples assigned to  $\mu_K$

$$\mu_K = \frac{1}{|\{i: z_i = K\}|} \sum_{i: z_i = K} \phi(x_i)$$

□ → available



K-means:

$$\text{Obj: } \min_z \min_{\mu} \text{loss}(z, \mu)$$

K mean

DECENT

\* ya data may say do e.g randomly pick Krain & initialize them as your centroids.

Date \_\_\_\_\_

Initialize (randomly)

$u_1, u_2, u_3, \dots, u_k$   
For  $t = 1, 2, \dots, T$

Step 1: set assignments  $z$  given  $u$

Step 2: set centroids  $u$  given  $z$

e.g

\* Input:  $D_{\text{train}} = \{0, 2, 10, 12\}$   
//  $D_{\text{train}} = \{(1, 2), (3, 1), (5, 2)\}$  2 features

Output:  $K = 2$   $u_1, u_2 \in \mathbb{R}$

Iteration 1:  $u_1 = 0, u_2 = 2$

Step 1:  $z_1 = 1, z_2 = 2, z_3 = 2, z_4 = 2$

Step 2:  $u_1 = 0, u_2 = \frac{2+10+12}{3} = 8$

DECENT

$u_1 = 0, u_2 = 8$

Iteration 2:

Step 1:  $z_1 = 1, z_2 = 1, z_3 = 2, z_4 = 2$

Step 2:

$u_1 = \frac{0+2}{2} = 1, u_2 = \frac{10+12}{2} = 11$

$u_1 = 1, u_2 = 11$

Iteration 3:

Step 1:  $z_1 = 1, z_2 = 1, z_3 = 2, z_4 = 2$

Step 2:

$u_1 = \frac{0+2}{2} = 1, u_2 = \frac{10+12}{2} = 11$

$u_1 = 1, u_2 = 11$

Since centroid did not change so our algorithm stops giving us 2 clusters.

DECENT

Date \_\_\_\_\_

$$\begin{array}{r} 4.3 \\ 3 \overline{)13} \\ \underline{-12} \\ 10 \end{array}$$

e.g.  $D_{\text{train}} = \{(0,1), (1,2), (5,6), (6,3)\}$

Output  $K = 2$ Run K-means with  $\mu_1 = (0,1)$ ,  $\mu_2 = (1,2)$ Iteration 1:  $\mu_1 = (0,1)$ ,  $\mu_2 = (1,2)$ 

Step 1:

$$z_1 = 1, z_2 = 2, z_3 = 2,$$

$$z_4 = 2$$

Step 2:

$$\mu_1 = (0,1), \mu_2 = \frac{(12, 13)}{3}$$

$$\mu_1 = (0,1), \mu_2 = (4, 4.3)$$

Iteration 2:  $\mu_1 = (0,1)$ ,  $\mu_2 = (4, 4.3)$ 

Step 1:

$$z_1 = 1, z_2 = 1, z_3 = 2,$$

$$z_4 = 2$$

DECENT

Date \_\_\_\_\_

Step 2:

$$\mu_1 = \frac{(1, 3)}{2}$$

$$\mu_2 = \frac{(11, 11)}{2}$$

$$\mu_1 = (0.5, 1.5), \mu_2 = (5.5, 5.5)$$

Iteration 3:

LECTURE # 12

14<sup>th</sup> Nov, 24

## STATE BASED MODELS:

First type of this model is :-

SEARCH PROBLEMS:

Farmer, goat, wolf, cabbage.

- want to cross the river

(one boat - can carry two)

Reflex Models:

- Features + architecture
- Modeling

- Based on data learning

- Query/Testing

Inference

DECENT

Date \_\_\_\_\_

Objective: cross with minimum no. of crossings

Constraints: Goat x wolf x  
Goat x cabbage x

• Applications:

Problem-1: route planning

Objective: Shortest, fastest, more scenic view.

Actions:

- go straight - turn left - turn right  
Source                      Destination  
A                                  D

∴ My solution would be a combination of sequence/actions.

Solution: sequence of actions

\* Actions have consequences on future actions. DECENT

Date \_\_\_\_\_

Problem-II: Robot Motion Planning

Objective: fastest, safest, energy efficient, most expressive

Actions:

- Translation - rotation

Problem-III: Solving Puzzles (e.g. Rubik's cube)

Objective: reach a certain configuration

Actions: move pieces

Search Problem vs Reflex Models:

RM:

Classification  $x \rightarrow \boxed{If} \rightarrow$  Single action (output)

SP / State Based Models:

$x \rightarrow \boxed{If} \rightarrow$  sequence of actions.  
(e.g.  $a_2, a_3, a_1$  -) DECENT

Date \_\_\_\_\_

- In reflex models outputs are independent but in this model actions have consequences and are dependent.

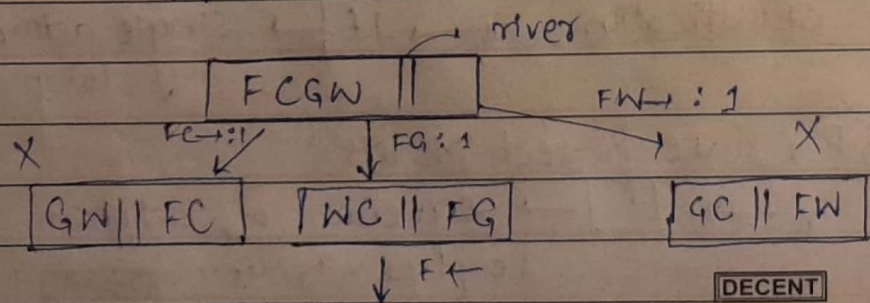
- R/M  $\rightarrow$  fixed computation
- SP  $\rightarrow$  sequence of actions

example: back to the goat problem

Approach: Tree Search (what if Tree)

Actions: left to right or right to left movements

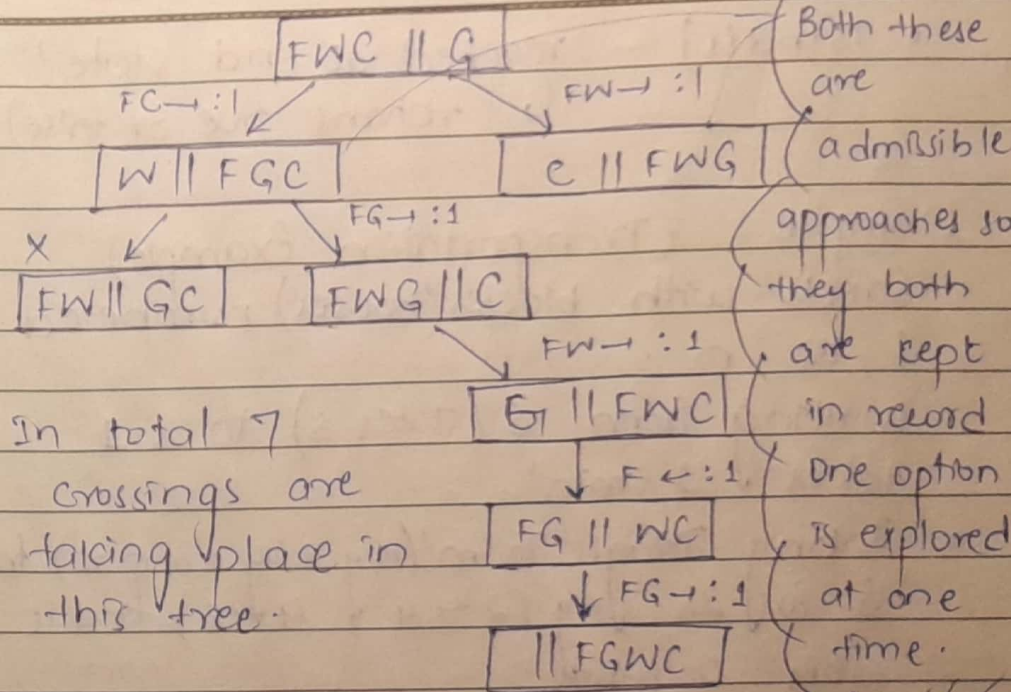
- Farmer  $\rightarrow$  or  $\leftarrow$  Farmer
- FC  $\rightarrow$  or  $\leftarrow$  FC
- FG  $\rightarrow$  or  $\leftarrow$  FG
- FW  $\rightarrow$  or  $\leftarrow$  FW



DECENT

: 1  $\rightarrow$  this represents a complete crossing.

Date \_\_\_\_\_



In total 7 crossings are taking place in this tree.

Formal Definition (Model) of Search Problem:

- $S_{start}$  — starting state
- Actions (s) — possible actions
- $Cost(s, a)$  — action cost
- $Succ(s, a)$  — successor state (Next state)

state spy  
bathing we learn  
look say  
actions perform  
for  
state hon

DECENT

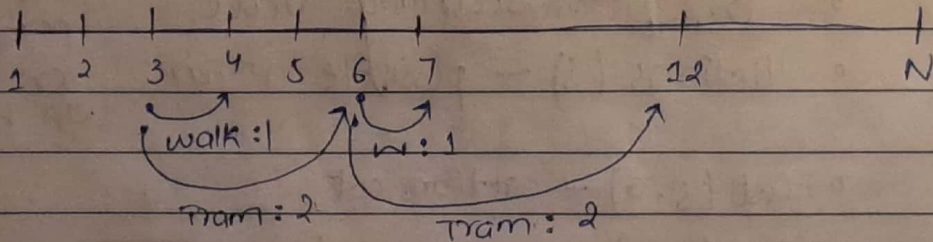
Date \_\_\_\_\_

- IsEnd(s) - reached at end state?  
↳ (returns true or false)

Example: (Transportation Example)

- Streets with blocks(houses) numbered 1 to n.
- Walking from s (states s) to s+1 takes 1 min.
- Taking Magic Tram (Any daily transport) from s to 2s e.g. (2 → 4, 4 → 8) and it takes 2 mins.

Question: How to travel from 1 to n in the least time?



DECENT

Date \_\_\_\_\_

Approach: Tree Search

Actions:

- $S_{start}$ : Block 1

LECTURE # 13

19<sup>th</sup> Nov, 24

Tuesday :- 26<sup>th</sup> Nov, 24 (Mid)

- Successor function: At any state if I walk then s+1 or train then 2s.

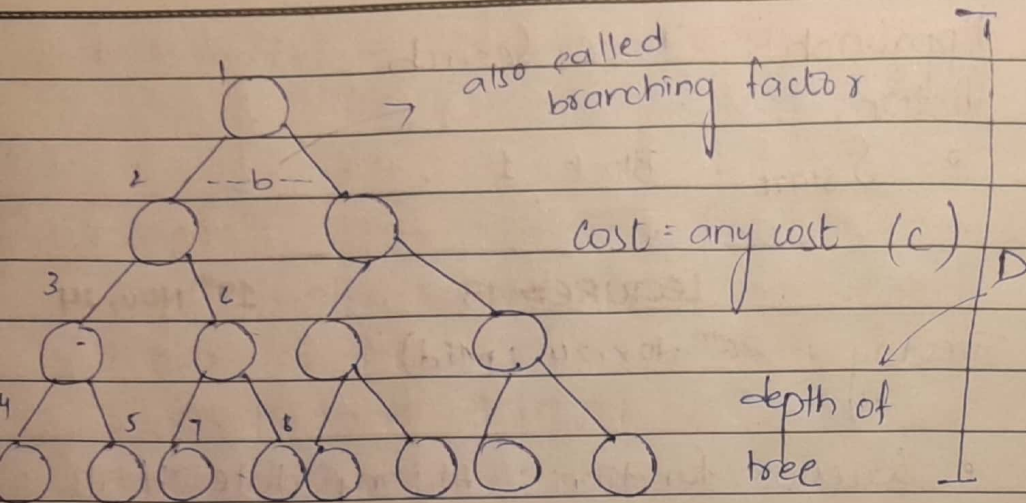
Different Algorithm To Solve the above problem

BACKTRACKING SEARCH:-

- depth first search.
- all possibilities are explored till the end pay solution deti. all nodes/states are explored.
- b - represents no. of actions per state.

DECENT

Date \_\_\_\_\_



Analysis :

space complexity (memory) :  $O(D)$   
 Time complexity (time) :  $O(b^D)$

\* if problem size increases then this algorithm is not useful. start state

Algorithm :

Def backtrackSearch( $s$ , Path)  
 if isEnd( $s$ ) : update minimum cost  
 path

DECENT

Date \_\_\_\_\_

for each action  $a \in \text{Action}(s)$   
 | Extend path success( $s, a$ ) and cost( $s, a$ )  
 | call BacktrackingSearch(success( $s, a$ ), path)  
 end

return minimum cost path,

→ Jo minimum wala hai wo apni pass reh lain baki ignore kr dein.

→ self shows the object of class  
 → list return ho gi → action - new state, cost

→ history → kis path se ho ky aye hai.  
 → cost → infinity used to compare to other.

DEPTH FIRST SEARCH:-

° Assumption is for every action

DECENT

Date \_\_\_\_\_

cost is 0.

- Idea: Backtracking + stop where first end state.

- Basically just searching whether a solution exists or not.

- Whenever the solution occurs the algorithm is stopped without searching for better solution.

time:  $O(2^D)$

memory:  $O(D)$

- theoretically same complexity as backtracking search but practically much less than that backtracking.

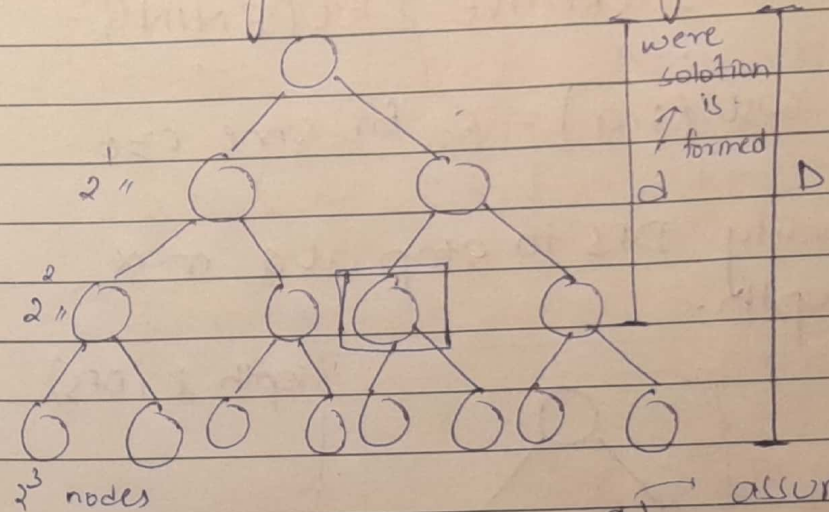
## BREADTH FIRST SEARCH:-

Assume cost (s,a) = c for some  $c \geq 0$

DECENT

Date \_\_\_\_\_

- level by level searching.



memory:  $O(b^D)$

time:  $O(b^D)$

assuming that solution is found first can be  $D$  too.

Table:

|              | cost       | Space    | Time     |
|--------------|------------|----------|----------|
| Backtracking | Any        | $O(D)$   | $O(b^D)$ |
| DFS          | 0          | $O(D)$   | $O(b^D)$ |
| BFS          | $c \geq 0$ | $O(b^D)$ | $O(b^D)$ |
| DFB-ID       | $c \geq 0$ | $O(D)$   | $O(b^D)$ |

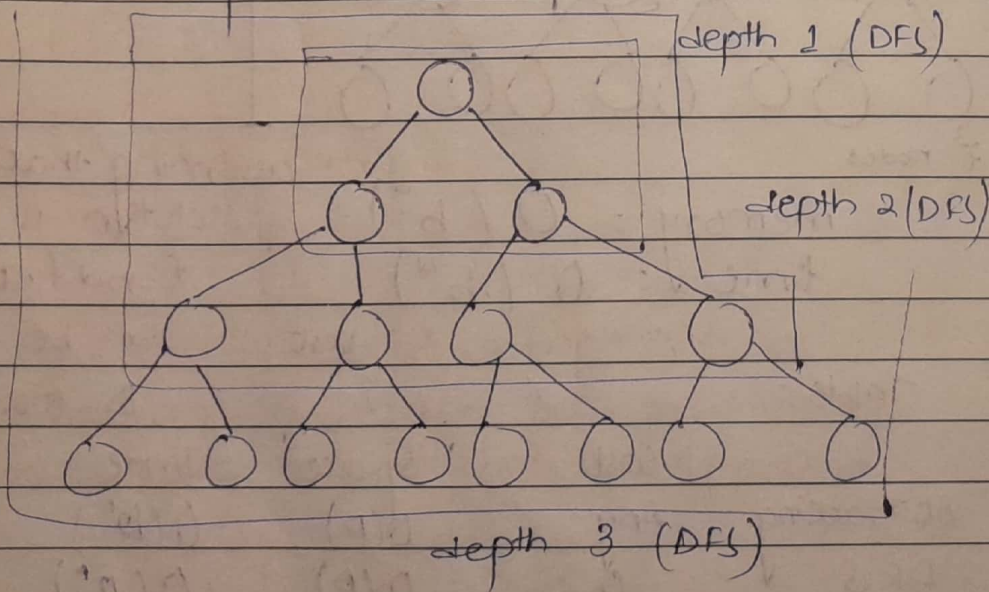
best

DECENT

Date \_\_\_\_\_

## DFS WITH ITERATIVE DEEPENING:-

- Assume cost  $(s, a) = c$  for some  $c \geq 0$
- Idea :
  - Modify DFS to stop at a max depth.



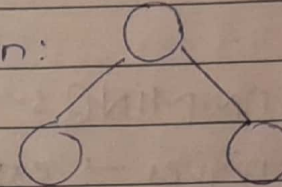
- we call DFS for max depth = 1, 2, ...  
DFS on  $d$  levels : is

DECENT

Date \_\_\_\_\_

there a solution with depth  $d$  ?

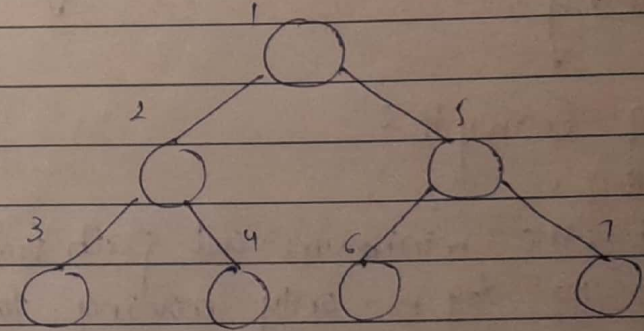
1<sup>st</sup> iteration:



DFS with  $d=1$

- If we find sol here programme stops.

2<sup>nd</sup> iteration:



3<sup>rd</sup> iteration:

//

//

Complexities:

memory -  $O(d)$

time  $\downarrow O(b \cdot b^d)$   
very insignificant

DECENT

## LECTURE # 14

Date 21<sup>st</sup> Nov, 24

All the previous had exponential time complexity.

### DYNAMIC PROGRAMMING:-

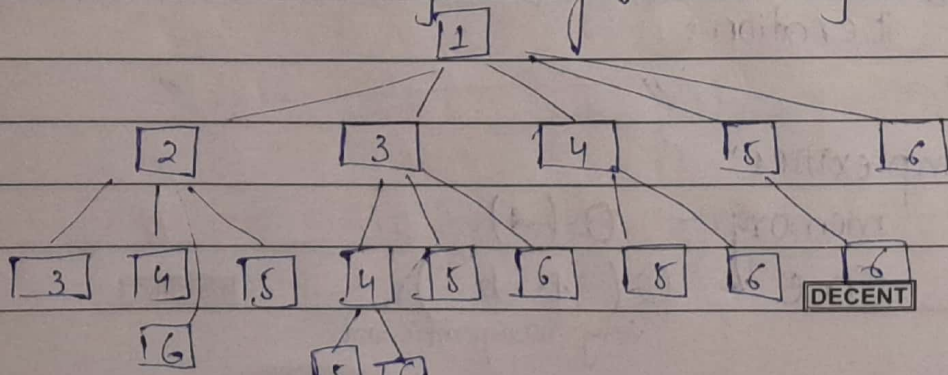
- Tree based techniques  $\rightarrow$  exponential in time

That's why we move towards dynamic programming.

Motivating Example:

Route Finding.

- Find the minimum cost path from city 1 to city  $n$ , only moving forward. It costs  $C_{ij}$  to go  $i$  to  $j$ .



Date \_\_\_\_\_

- By this the state on which you are currently standing gives the shortest or next state.

- sub-trees repeat no rhy hain.
- In DP we look at:

Future Cost( $s$ ) - Cost from state  $s$  to end state.

$\text{succ}(s, a) \rightarrow \text{cost}(s, a), s'$   
 $\swarrow$  cost of the moving  $\searrow$  Next state

$s$

$s'$

Send

$$\text{Future Cost}(s) = \text{cost}(s, a) + \text{Future Cost}(s')$$

$$\text{Future Cost}(s) = \begin{cases} 0 & \text{if } \text{end}(s) = T \\ \min_a \{ \text{cost}(s, a) + \text{Future Cost}(s') \} & \text{otherwise} \end{cases}$$

- Now we define algorithm of DP.

$$s' = \text{succ}(s, a)$$

DECENT

Date \_\_\_\_\_

DP- Algorithm:

(if repeated sub-tree occur)

Def  $DP(s)$

if already computed for  $s$ , return cache answer

if  $IsEnd(s) = \bar{1}$  return solution/zero

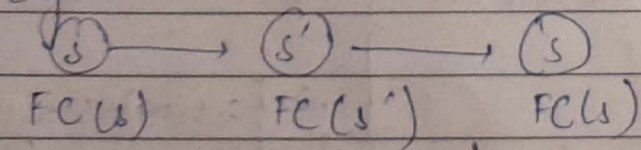
for each action  $a \in Actions(s)$ :

- - - - -

(same as back-tracking)

Assumptions:-

Acyclicity: The state graph defined by action  $(s)$  and  $succ(s, a)$  is acyclic



• This results in infinite loop, bcz for cost of  $s$  we need cost of  $s'$  and if  $s'$  in turn need

DECENT

3Blue1Brown (Keya)

Date \_\_\_\_\_

$s$  then it becomes an infinite loop.  
Code: Transportation prob.

- In DP:

• state: The state is summary of all past actions sufficient to choose the future actions optimally.

Handling Additional Constraints:

Travel from city 1 to city  $n$ , only move forward  $C_{ij}$ .

constraints: Can't visit three odd cities in a row.

• previously: state: current city.

• But for this: 1 of the solutions can be.

→ state: (# of odd cities, current city)

half the total of population  $\leftarrow \frac{N}{2}$

DECENT

- (idhar after calculation discard ho jayain gay). (Nahta le) eg

Date \_\_\_\_\_

State space:  $N^2$  ( $N/2 * N$ ) → This much combinations of cities will be made.

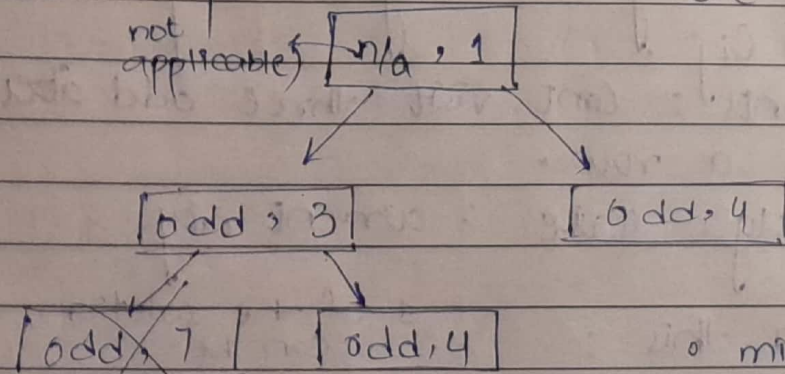
\* Another solution more optimal to reduce state space can be:

State:  $(\min(\# \text{ of odd cities}, 3), \text{current city})$

↓  
3

N

State Space:  $3 \times N \rightarrow O(N)$



- min steps?
- min cost?

• Actions

DECENT

Date \_\_\_\_\_

Uniform Cost Search Algorithm:

- DP can't work for cyclic graphs
- Then we use this:

Assumptions:

- All actions cost are non-negative
- $\text{cost}(s, a) \geq 0$

Idea:

UCS enumerates states in order of increasing past cost.

High Level Strategy: (The previous all cost while we maintain  $\sqrt{3}$  sets: you stand on a certain

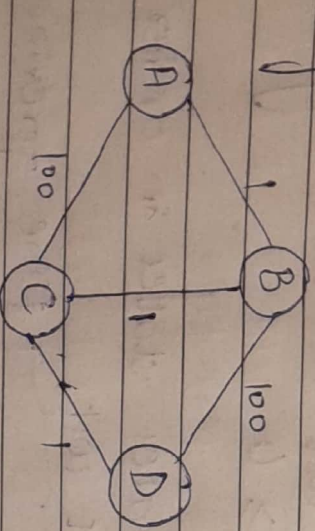
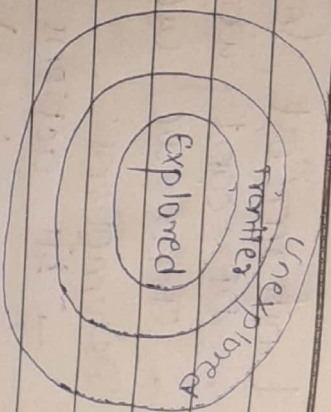
- Explored: States we have state.) found the optimal path to.
- Frontier: States we have seen but still figuring out how to get there (by minimum cost).
- Unexplored: States we have

DECENT

Date \_\_\_\_\_

not seen yet.

- This can work on large / infinite graphs



start city: A

Destination: D

path cost city

unexplored

①

frontier

A, D

explored

during

②

B, 1

④

B, 1

stand

③

C, 100

⑧

C, 2

they

⑤

D, 101

⑩

D, 13

all will

be here

Minimum

cost = 3

- path cost is kept in increasing order

DECENT

\* already explored sets are not added in frontier again.

Date \_\_\_\_\_

A → B → C → D

cost: 3

Algorithm:

Add start state (s) to frontier

Repeat until frontier is empty

Remove s with smallest priority

if is end: return D

Add s to explored set

for each a ∈ action(s)

Get successor s' → succ(s, a)

if s' is already explored: continue

update frontier with s' and priority P + cost(s, a)

unexplored

DECENT